EASY GUIDE · 4-BIT ALU PROJECT

Verilog and Vivado, explained simply

This guide explains your 4-bit ALU project in plain words: what Verilog is, what each line of the code means, what Vivado does, and exactly which buttons to click in Vivado to simulate the ALU and see its circuit. No scripts and no command line are needed.

Short answer: you do not need run.sh

`run.sh` was only a shortcut for *my* computer. I tested your design with a free simulator called Icarus Verilog, which is used by typing commands, and `run.sh` just saved me typing them. Vivado has its own simulator built in, and clicking **Run Behavioral Simulation** does the same job. So `run.sh` has been **removed**. You only need the ten `.v` files.

You need	Vivado (any recent version, the free edition is fine) and the 10 `.v` files
Design files (9)	the ALU itself — they become hardware
Testbench (1)	`alu_4bit_tb.v` — only for testing in simulation
Board	not needed for simulation, schematic or synthesis

1 Four words to know
2 Why run.sh is gone
3 Your files, and where they go in Vivado
4 Verilog in easy words
5 How the ALU works
6 Step by step in Vivado
7 If something goes wrong
8 Putting it on a board (optional)
9 One-page checklist

1 Four words to know

WORD	WHAT IT MEANS	EVERYDAY COMPARISON
Verilog	A language for *describing a circuit*. You write what the wires and gates do, not steps for a CPU to run.	The written plan of a building
Simulation	Testing the Verilog on the computer: give inputs, check outputs. Nothing physical is made.	Checking the plan on paper
Synthesis	Turning the Verilog into real logic blocks (LUTs and flip-flops) that exist on a chip.	Turning the plan into bricks
Vivado	AMD/Xilinx software that does all of this: edit, simulate, synthesise, and program an FPGA board.	The construction company

An **FPGA** is a chip whose logic can be rewired by loading a file (a *bitstream*). Vivado makes that file from your Verilog. For a lab or project report, most of the time you only need **simulation** (to prove the ALU works) and the **schematic** (to show the circuit).

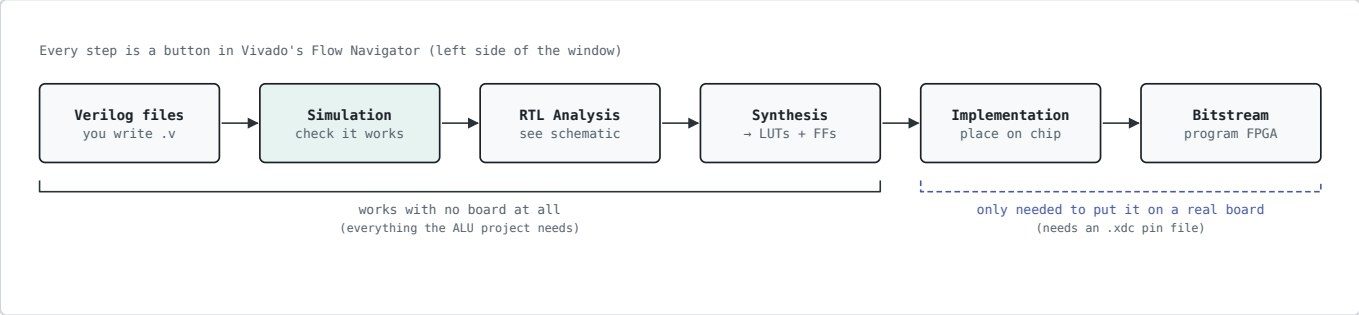


Figure 1. The Vivado flow. You move left to right. For this ALU project the first four steps are enough; the last two are only for a real board.

2 Why run.sh is gone

A Verilog project always has two jobs: **compile** the files (check the code and connect the modules) and **run** the testbench. With a command-line simulator you type those jobs yourself; `run.sh` was just a text file holding the two commands. Vivado does both jobs when you click one button, so the file has no purpose in a Vivado project.

WHAT RUN.SH DID	WHAT YOU DO IN VIVADO INSTEAD
Listed the 9 design files + testbench to compile	`Add Sources` once — Vivado remembers the files in the project
`iverilog …` (compile)	Happens automatically when you start simulation
`vvp …` (run the testbench)	`Run Behavioral Simulation` then `Run All`
Printed results in a terminal	Results print in the `Tcl Console` at the bottom of Vivado
Wrote a .vcd file for GTKWave	Vivado shows waveforms itself in the `Wave` window

Two small changes were also made so the files fit Vivado better:

- Every design file now starts with `` `timescale 1ns / 1ps `` (Vivado expects this; see 4.9).
- The testbench no longer writes a `.vcd` waveform file, because Vivado draws the waveforms for you. The testbench also has instructions at the top for running it in Vivado.

The ALU itself did not change. The same testbench still passes all 4,096 test cases, and a strict lint check with Verilator found no problems in the design files.

3 Your files, and where they go in Vivado

Vivado sorts files into groups. **Design Sources** are turned into hardware. **Simulation Sources** are only used for testing. **Constraints** (`.xdc`) connect signals to board pins; you do not need any for this project unless you use a board.

FILE	ADD AS	WHAT IT IS, IN ONE LINE
`alu_4bit.v`	Design source (**top**)	The whole ALU: plugs all the other blocks together
`control_unit.v`	Design source	Reads the opcode and tells the other blocks what to do
`adder_subtractor_4bit.v`	Design source	Adds or subtracts two 4-bit numbers
`full_adder.v`	Design source	Adds three bits; used 4 times by the adder
`logic_unit_4bit.v`	Design source	AND, OR, XOR, NOT
`shifter_4bit.v`	Design source	Shift left / shift right
`muldiv_unit.v`	Design source	Multiply and divide by shifting, one step per clock
`result_mux.v`	Design source	Picks which answer goes to the output
`flag_unit.v`	Design source	Makes the carry, zero, overflow and negative flags
`alu_4bit_tb.v`	Simulation source (**top**)	The tester: feeds inputs and checks every answer

Think of the files as LEGO blocks. Each file is one block with pins. `alu_4bit.v` is the base plate that snaps them together, and the testbench is the person testing the finished model. After you add the files, Vivado's **Sources** window shows this tree:

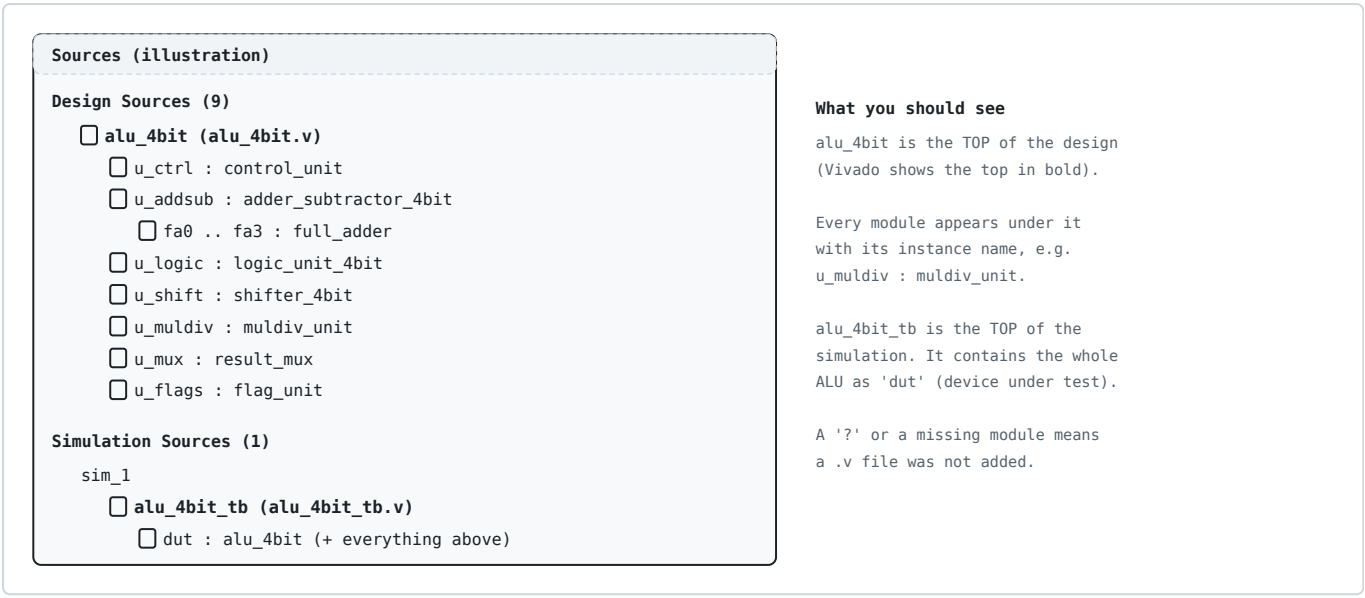


Figure 2. What the Sources window should look like. Drawn illustration, not a screenshot. Names after the colon are module names; names before it are instance names used inside alu_4bit.v.

4 Verilog in easy words

Every idea below appears in your files. Read the code on the left, then the plain explanation on the right.

4.1 module — a chip with pins

```
module full_adder (
    input  wire a,
    input  wire b,
    input  wire cin,
    output wire sum,
    output wire cout
);
    ...
endmodule
```

A **module** is one block of hardware, like a chip. The list in brackets is its **pins** (called *ports*). `input` pins receive signals, `output` pins send them. Everything between `module` and `endmodule` is what is inside the chip. Each of your files holds exactly one module with the same name as the file.

4.2 Buses — several wires with one name

```
input  wire [3:0] a,        // 4 wires: a[3] a[2] a[1] a[0]
output wire [7:0] result,   // 8 wires
wire   [3:0] sum;           // internal 4-bit wire
```

`[3:0]` means “4 wires numbered 3 down to 0”. Bit 3 is the biggest (worth 8) and bit 0 the smallest (worth 1). `a[2]` means just one of those wires, and `a[3:1]` means wires 3, 2 and 1.

4.3 wire and reg

```
wire [3:0] sum;     // driven by assign or a module
reg  [3:0] acc;     // given a value inside always
```

A **wire** is a real connection; it always shows whatever is driving it. A **reg** is a signal that gets its value inside an `always` block. Careful: `reg` does *not* always mean memory. In `result_mux.v` the output is a `reg` but the circuit is plain logic. Only regs written on a clock edge (4.6) become flip-flops.

4.4 assign — a permanent connection

```
// logic_unit_4bit.v
assign and_out = a & b;
assign or_out  = a | b;
assign xor_out = a ^ b;
assign not_out = ~a;
```

`assign` connects a wire to some logic for ever. It is not a step that runs once; whenever `a` or `b` changes, the outputs change straight away, just like real gates. The symbols are `&` AND, `|` OR, `^` XOR, `~` NOT.

4.5 always @(*) — logic written like a program

```
// result_mux.v
always @(*) begin
    case (opcode)
        4'b0000, 4'b0001: result = {4'b0000, sum};
        4'b0010:          result = {4'b0000, and_out};
        ...
        default:          result = 8'b0000_0000;
    endcase
end
```

`always @(*)` means “whenever any input changes, work this out again”. It lets you use `case` and `if`, which is easier to read than many `assign`s, but the result is still plain logic with no clock. Here it builds a **multiplexer**: the opcode chooses which answer goes out. `4'b0010` means a 4-bit binary number 0010. The `default` line makes sure the output always has a value.

4.6 always @(posedge clk) — flip-flops and memory

```
// muldiv_unit.v (short version)
always @(posedge clk or posedge rst) begin
    if (rst)
        count <= 2'd0;
    else if (running)
        count <= count + 2'd1;
end
```

This block only acts at the **rising edge of the clock** (or when reset goes high). Between edges the values are held, so this is where **flip-flops** (memory) come from. `muldiv_unit.v` is the only file with a clock, because multiply and divide take several steps.

> **The = and <= rule.** Use <= (non-blocking) inside clocked blocks, so every flip-flop updates together at the clock edge. Use = (blocking) inside `always @(*)` blocks. Your files follow this rule everywhere.

4.7 Using one module inside another (instantiation)

```
// alu_4bit.v
logic_unit_4bit u_logic (
    .a(a),  .b(b),
    .and_out(and_out), .or_out(or_out),
    .xor_out(xor_out), .not_out(not_out)
);
```

This places one copy of the `logic_unit_4bit` block inside the ALU and names the copy `u_logic`. Each `.pin(signal)` connects a pin of the block to a wire in the ALU: pin `.a` is connected to the ALU's wire `a`. This is how all the files are linked together, and it is why Vivado shows the tree in Figure 2.

4.8 Joining bits with { }

```
// shifter_4bit.v
assign shl = {a[2:0], 1'b0};  // 1011 -> 0110
assign shr = {1'b0, a[3:1]};  // 1011 -> 0101
```

Curly brackets join bits side by side into one bigger value. For shift left, the lower three bits of `a` are taken and a 0 is added on the right, so every bit moves one place left. No gates are needed; it is just wiring.

4.9 `timescale

```
`timescale 1ns / 1ps
```

This tells the simulator that `#1` means 1 nanosecond, measured to a precision of 1 picosecond. It only matters in simulation (delays like `#5` in the testbench). Hardware ignores it. Vivado prefers all files to have the same line, so every file now has it.

4.10 The testbench — a virtual tester

```
module alu_4bit_tb;          // no pins at all
    reg  [3:0] a, b, opcode;  // tester drives these
    wire [7:0] result;        // tester watches this
    alu_4bit dut ( ... );     // the ALU being tested
    always #5 clk = ~clk;     // clock: flips every 5 ns
    initial begin
        opcode = 4'b0000; a = 5; b = 3;
        #1;                                  // wait 1 ns
        $display("result = %d", result);    // print
        $finish;                             // stop
    end
endmodule
```

The testbench is **not hardware**. It is like a lab bench with switches and a meter. It has no pins, uses `reg` for the inputs it sets, and `wire` for the outputs it reads. `#5` means “wait 5 ns”. `initial` runs once from time 0. `$display` prints text to Vivado's Tcl Console, and `$finish` ends the simulation. The name `dut` means “device under test”.

5 How the ALU works

The ALU takes two 4-bit numbers, `a` and `b`, and a 4-bit `opcode` that picks one of ten operations. All the blocks work on `a` and `b` at the same time, and `result_mux` lets only the chosen answer through.

OPCODE	DOES	EXAMPLE (A = 12, B = 10)	TIME
`0000`	ADD	12 + 10 = 22 → 4 bits show 6, carry = 1	instant
`0001`	SUB	12 − 10 = 2	instant
`0010` / `0011` / `0100`	AND / OR / XOR	1100 & 1010 = 1000 …	instant
`0101`	NOT a	~1100 = 0011	instant
`0110` / `0111`	shift left / right	1100 → 1000 / 0110	instant
`1000`	MUL	12 × 10 = 120 (8-bit result)	5 clocks
`1001`	DIV	12 ÷ 10 = 1 remainder 2	5 clocks

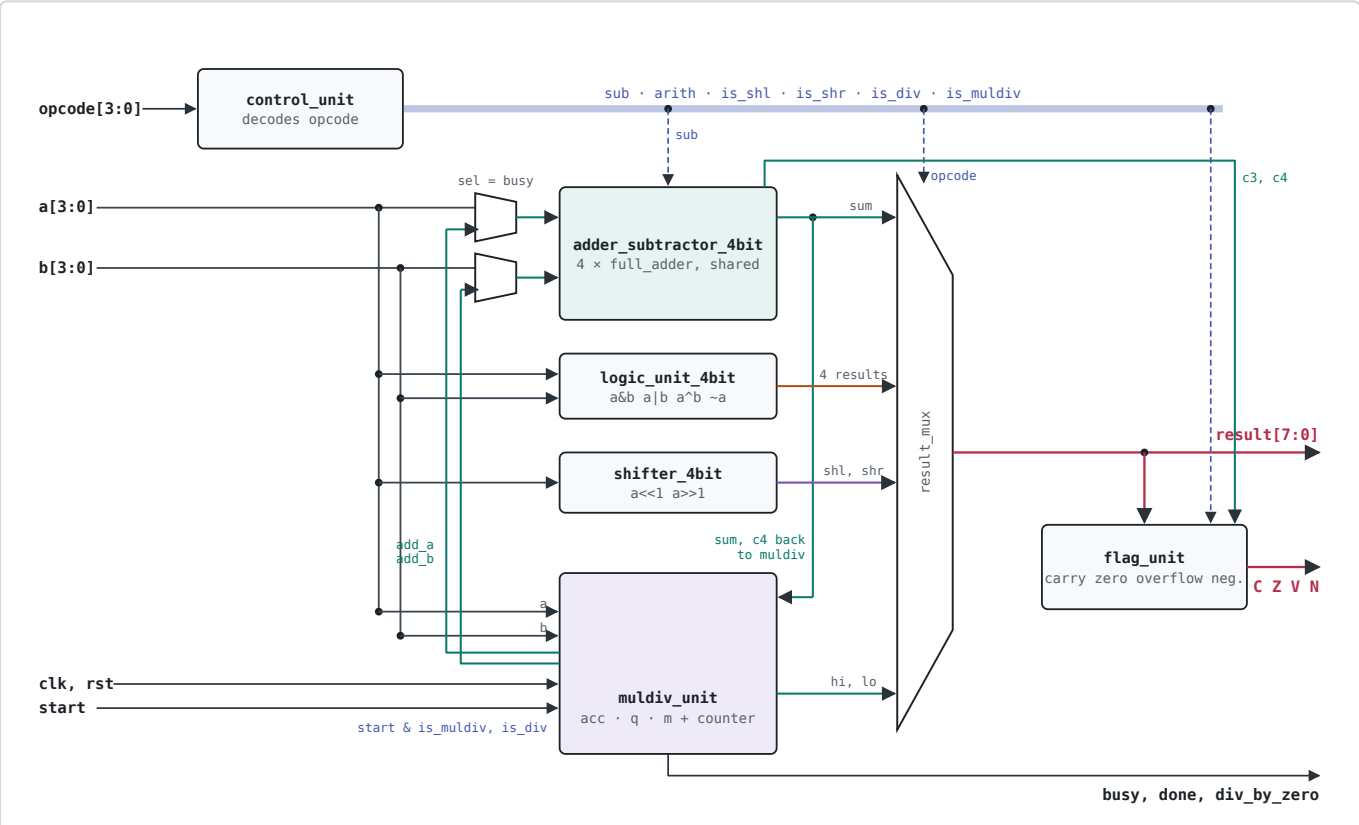


Figure 3. Inside alu_4bit.v. Each box is one of your files. The adder is shared: normally it does ADD and SUB, and while a multiply or divide is running, muldiv_unit borrows it.

Multiply and divide in easy words

Multiply works like long multiplication at school, but in binary. Look at one bit of `b` at a time. If the bit is 1, add `a` to the running total; if it is 0, add nothing. Then shift everything one place. After 4 bits (4 clocks) the answer is complete. **Divide** works like long division: bring down one bit, check whether `b` fits, subtract if it does and write 1 in the answer, otherwise write 0. Both

use the ALU's one adder instead of building a big multiplier and divider, which keeps the circuit small.

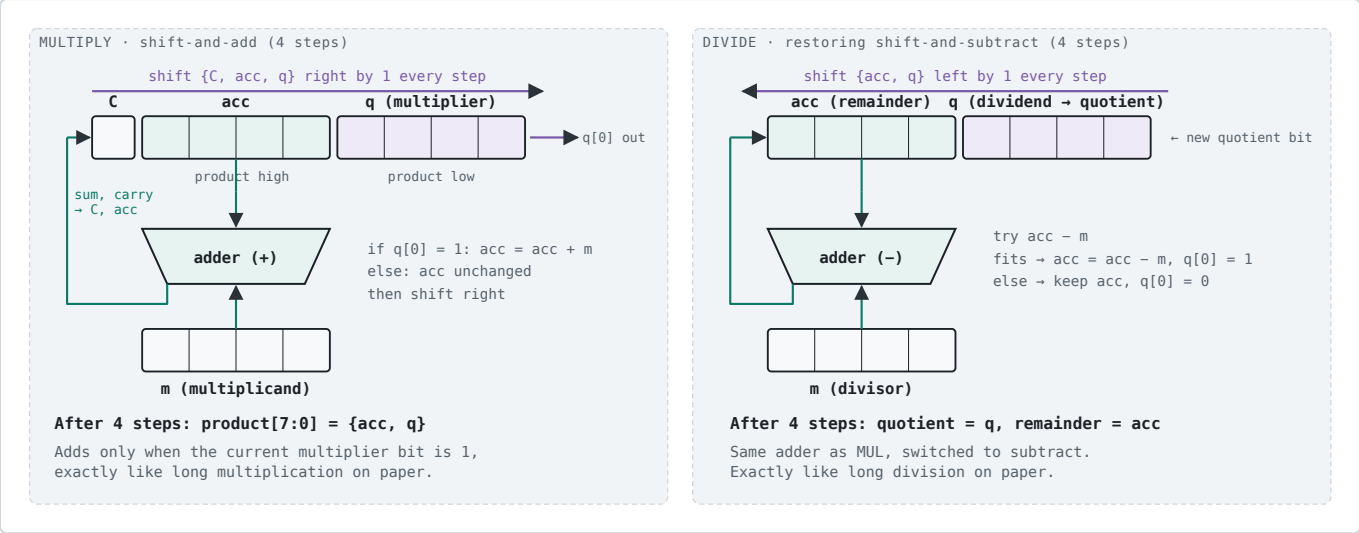


Figure 4. The multiply and divide steps. acc, q and m are 4-bit registers (flip-flops) inside muldiv_unit.v.

How to use MUL/DIV: set `a`, `b` and the opcode, make `start` = 1 for one clock, then wait until `done` = 1 and read `result`. For DIV, `result[3:0]` is the quotient and `result[7:4]` the remainder. The full theory is in the main report.

6 Step by step in Vivado

Button names are from recent Vivado versions (2019 and later); older versions look almost the same.

A. Make the project

1 **Create a new project**

Open Vivado → `Create Project` → `Next`. Give it a name, for example `alu_4bit`, and pick a folder with **no spaces** in the path.

2 **Project type**

Choose `RTL Project`. You may tick `Do not specify sources at this time` and add them later, or add them now (steps 4 and 5 show the same thing).

3 **Choose a part**

If you have a board, pick it on the `Boards` tab (for example Basys 3). If not, pick any part, for example `xc7a35tcpg236-1`. For simulation it does not matter. Click `Finish`.

B. Add the files

4 **Add the 9 design files**

In the Flow Navigator on the left: `PROJECT MANAGER` → `Add Sources` → `Add or create design sources` → `Add Files`. Select all nine design files (everything except `alu_4bit_tb.v`). Tick `Copy sources into project` and click `Finish`.

5 **Add the testbench**

`Add Sources` again → `Add or create simulation sources` → `Add Files` → `alu_4bit_tb.v` → `Finish`.

6 **Check the tops**

In the Sources window, `alu_4bit` should be bold under Design Sources, and `alu_4bit_tb` should be bold under Simulation Sources → sim_1 (Figure 2). If a different file is bold, right-click the right one → `Set as Top`.

C. Simulate — prove the ALU works

7 **Start the simulation**

`SIMULATION` → `Run Simulation` → `Run Behavioral Simulation`. Vivado compiles all the files (this is what `run.sh` used to do) and opens the simulation view.

8 **Click Run All**

Vivado stops after **1000 ns** by default, but this testbench needs about **35 µs**. Click the `Run All` button (▶ with a line, or press F3), or type `run all` in the Tcl Console and press Enter.

9 **Read the result**

Look at the `Tcl Console` tab at the bottom. You should see the table of examples and, at the end, the PASS line:

```
 Op   code |  A   B |  result        | flags / notes
----------+--------+----------------+------------------------------
ADD  0000 |  5   3 | 00001000 (  8) | C=0 Z=0 V=1 N=1
ADD  0000 | 15   1 | 00000000 (  0) | C=1 Z=1 V=0 N=0
SUB  0001 |  5   3 | 00000010 (  2) | C=1 Z=0 V=0 N=0
SUB  0001 |  3   5 | 00001110 ( 14) | C=0 Z=0 V=0 N=1
AND  0010 | 12  10 | 00001000 (  8) | C=0 Z=0 V=0 N=1
OR   0011 | 12  10 | 00001110 ( 14) | C=0 Z=0 V=0 N=1
XOR  0100 | 12  10 | 00000110 (  6) | C=0 Z=0 V=0 N=0
NOT  0101 | 12   0 | 00000011 (  3) | C=0 Z=0 V=0 N=0
SHL  0110 | 11   0 | 00000110 (  6) | C=1 Z=0 V=0 N=0
SHR  0111 | 11   0 | 00000101 (  5) | C=1 Z=0 V=0 N=0
MUL  1000 | 13  11 | 10001111 (143) | product = 143, 5 cycles
MUL  1000 | 15  15 | 11100001 (225) | product = 225, 5 cycles
DIV  1001 | 13   4 | 00010011       | quotient = 3, remainder = 1, dbz=0, 5 cycles
DIV  1001 | 15   3 | 00000101       | quotient = 5, remainder = 0, dbz=0, 5 cycles
DIV  1001 |  9   0 | 10011111       | quotient = 15, remainder = 9, dbz=1, 5 cycles

PASS: 3584 single-cycle checks + 512 MUL/DIV checks, 0 errors
```

> **Tip — run all automatically.** Settings → Simulation → Simulation tab → set `xsim.simulate.runtime*` to `all` (or `40us`). Then the whole test runs every time you start the simulation.

D. Look at the waveforms

10 See the top-level signals

The Wave window already shows the testbench signals: `clk`, `a`, `b`, `opcode`, `result` and the flags. Click Zoom Fit to see the whole run.

11 Add signals from inside the ALU

In the Scope panel open `alu_4bit_tb` → `dut` → `u_muldiv`. In the Objects panel, select `acc`, `q`, `count`, `busy` and `done`, right-click → Add to Wave Window.

12 Refill the new waves

New signals are empty until you run again: click Restart (⏮) and then Run All.

13 Show numbers nicely

Right-click a bus such as `result` → Radix → Unsigned Decimal (or Binary). Zoom into the first 200 ns to see the first multiply (13 × 11), as below.

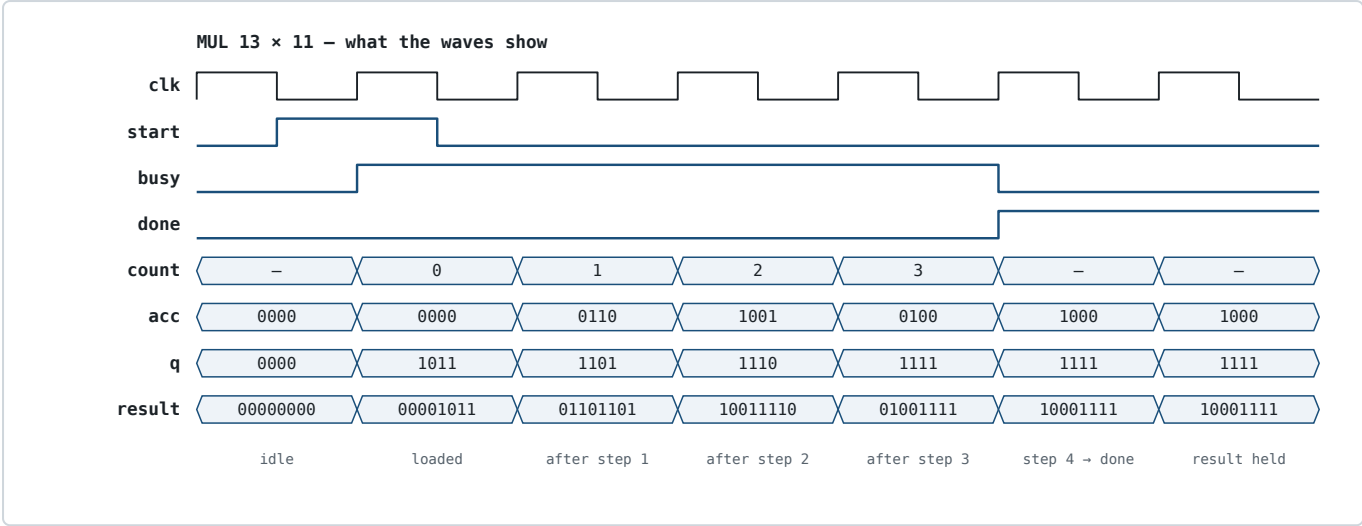


Figure 5. Multiply waveform. Drawn from our simulation of these files. In Vivado you will see the same values once you add acc, q, count, busy and done from u_muldiv.

E. See the circuit (schematic)

14 Open the elaborated design

RTL ANALYSIS → Open Elaborated Design → OK, then Schematic. You will see `alu_4bit` with its blocks (`u_ctrl`, `u_addsub`, `u_muldiv` …) connected like Figure 3. Double-click a block to look inside it. This picture is great for a report.

F. Synthesis — how big is it?

15 Run synthesis

SYNTHESIS → Run Synthesis. It needs no board or pin file. Wait for the “Synthesis successfully completed” message.

16 Read the size

Open Synthesized Design → Report Utilization. Vivado counts **LUTs** (small logic blocks) and **Registers** (flip-flops). You should see **18 registers**: the `acc`, `q` and `m` registers (12), the counter (2) and four control bits. The LUT count depends on your Vivado version and part.

> Vivado may show a few **warnings** during synthesis, for example about signals that never change. Warnings are normal. Only **errors**, shown in red, stop the flow.

7 If something goes wrong

WHAT YOU SEE	WHY	FIX
Simulation stops, no PASS line	Vivado stopped at 1000 ns	Click `Run All`, or set the run time to `all` (tip in step 9)
"module … not found" or a **?** in the Sources tree	A design file was not added	`Add Sources` → add the missing `.v` file
The wrong file is simulated	Wrong simulation top	Right-click `alu_4bit_tb` → `Set as Top`
Synthesis tries to use the testbench	Testbench added as a design source	Remove it and add it again as a simulation source
Added signals are empty in the Wave window	They were added after the run	`Restart` then `Run All`
You edited a file but nothing changed	Old simulation still open	`Relaunch Simulation` (circular arrow button)
Signals show **X** (red) or **Z** (blue)	X = unknown value, Z = nothing connected	Check the port connections and that `rst` was pulsed
"FAIL …" lines in the Tcl Console	A file was changed and a result is now wrong	Compare your file with the original
Implementation / bitstream error about unconstrained ports (UCIO, NSTD)	No pin file for a board	Only needed for a board: add an `.xdc` file (Section 8)

8 Putting it on a board (optional)

For simulation, the schematic and synthesis you do not need a board. To run the ALU on a real FPGA board you also need a **constraints file** (`.xdc`). It tells Vivado which physical pin each signal uses, for example "`a[0]` is switch 0" and "`result[0]` is LED 0". Pin names are different for every board, so the file must match your board. A typical board such as the Basys 3 has enough switches, buttons and LEDs:

ALU SIGNAL	BOARD PART	COUNT
`a`, `b`, `opcode`	switches	12
`rst`, `start`	push buttons	2
`clk`	board clock (for example 100 MHz)	1
`result` + `carry, zero, overflow, negative, div_by_zero, busy, done`	LEDs	8 + 7 = 15

Then run `Run Implementation` → `Generate Bitstream` → `Open Hardware Manager` → `Program Device`.

9 One-page checklist

☐ Create Project → RTL Project → pick a part or board

☐ Add Sources → design sources → the 9 design `.v` files (copy into project)

☐ Add Sources → simulation sources → `alu_4bit_tb.v`

☐ Sources window: `alu_4bit` is the design top, `alu_4bit_tb` is the simulation top

☐ Run Behavioral Simulation → **Run All**

☐ Tcl Console ends with: `PASS: 3584 single-cycle checks + 512 MUL/DIV checks, 0 errors`

☐ Wave window: add `u_muldiv` signals → Restart → Run All → set Radix

☐ Open Elaborated Design → Schematic (screenshot it for your report)

☐ Run Synthesis → Report Utilization (18 registers)

☐ Optional: board → add `.xdc` → Implementation → Bitstream → Program Device

Remember

Design sources become hardware. The **testbench** only tests. **Vivado replaces run.sh**: Add Sources lists the files, and Run Simulation compiles and runs them. Always press **Run All** so the whole test runs.